

**ETEC PROF CAMARGO ARANHA
ENSINO MÉDIO COM TÉCNICO INTEGRADO A DESENVOLVIMENTO DE
SISTEMAS (M-TEC)**

LHORRANY CERQUEIRA SILVA CARDOSO DE OLIVEIRA
MARIA CAROLINA SAYO OGASSAWARA FREITAS
MATHEUS MINAMI PACHECO
RICKELMY FEITOSA MALAFAIA
RODRIGO MATEUS ALVARENGA SILVA

DOCUMENTAÇÃO TÉCNICA DE SOFTWARE - OPEN SOUND

SÃO PAULO
2026

LHORRANY CERQUEIRA SILVA CARDOSO DE OLIVEIRA

MARIA CAROLINA SAYO OGASSAWARA FREITAS

MATHEUS MINAMI PACHECO

RICHELMY FEITOSA MALAFAIA

RODRIGO MATEUS ALVARENGA SILVA

DOCUMENTAÇÃO TÉCNICA DE SOFTWARE - OPEN SOUND

Documentação técnica de software
apresentada ao curso de Ensino Médio com
Técnico Integrado a Desenvolvimento de
Sistemas (M-TEC) da ETEC Prof. Camargo
Aranha, como material de apoio à avaliação
escolar.

Orientadora: Célide Tasso

SÃO PAULO

2026

SUMÁRIO

SUMÁRIO.....	3
1. INTRODUÇÃO.....	5
1.1 Objetivo do documento.....	5
1.2 Escopo do sistema.....	5
1.3 Convenções utilizadas.....	5
2. VISÃO GERAL DO PRODUTO.....	6
2.1 Descrição geral.....	6
2.2 Público-alvo.....	6
2.3 Posicionamento em relação a referências de mercado.....	6
2.4 Estado atual do projeto.....	6
3. ARQUITETURA DO SISTEMA.....	7
3.1 Visão macro.....	7
3.2 Diagrama textual de arquitetura.....	7
3.3 Stack tecnológica.....	8
3.4 Organização de diretórios.....	9
4. MODELAGEM DE DADOS.....	10
4.1 Tabela usuarios.....	10
Tabela 2 - Estrutura da tabela usuarios.....	10
Fonte: Elaborado pelos autores (2026).....	11
4.2 Tabela musicas.....	11
4.3 Tabelas de verificação temporária.....	11
Tabela 4 - Tabelas de verificação temporária.....	11
4.4 Tabelas de histórico, playlists, perfil e player.....	12
4.5 Relacionamentos e integridade.....	12
5.1 Cadastro com verificação em duas etapas (2FA).....	14
5.2 Login e emissão de token (JWT).....	14
5.3 Hash de senhas.....	15
5.4 Recuperação e redefinição de senha.....	15
5.5 Limitação de requisições (rate limiting).....	15
5.6 Armazenamento de arquivos.....	16
5.7 Exclusão de conta.....	16
6. REQUISITOS FUNCIONAIS (MVP).....	17
7. API REST - REFERÊNCIA DE ENDPOINTS.....	19
7.1 Autenticação e cadastro.....	19
7.2 Usuário e perfil.....	19
7.3 Músicas.....	20
7.4 Playlists e favoritos.....	21
7.5 Player e fila de reprodução.....	22
8. FRONTEND - ESTRUTURA, TAGS E COMPONENTES.....	23
8.1 Barra de navegação (<nav class="fixed-bar">).....	23

8.2 Conteúdo principal (<div class="main-ctt">)	23
8.3 Modais (<div class="modal">)	23
8.4 Player fixo (<div id="player-barra">)	24
8.5 Nomenclatura	24
9. FLUXOS DE USO (CASOS DE USO)	25
9.1 Cadastro de novo usuário	25
9.2 Login e recuperação de senha	25
9.3 Tornar-se artista e enviar música	25
9.4 Reprodução, favoritos e playlists	26
9.5 Gestão de perfil e conta	26
10. REQUISITOS NÃO FUNCIONAIS	27
11. INSTALAÇÃO E EXECUÇÃO (AMBIENTE DE DESENVOLVIMENTO)	28
11.1 Pré-requisitos	28
11.2 Passo a passo	28
11.3 Variáveis de ambiente necessárias	28
12. PERSPECTIVAS FUTURAS	30
13. REFERÊNCIAS	31

1. INTRODUÇÃO

1.1 Objetivo do documento

Este documento descreve, de forma técnica e organizada, o funcionamento do software OpenSound: sua arquitetura, modelagem de dados, regras de negócio, endpoints de API, estrutura de frontend e os requisitos considerados necessários para que o sistema evolua de um protótipo funcional (MVP) para uma versão apta a ser publicada. O documento serve tanto como referência para a equipe de desenvolvimento quanto como material de apoio para avaliação escolar.

1.2 Escopo do sistema

O OpenSound é uma aplicação web que permite que usuários criem conta, façam login autenticado e enviem (upload) arquivos de música, disponibilizando-os para reprodução por outros usuários cadastrados. O catálogo de músicas é público para navegação, mas a reprodução exige autenticação. O sistema é inspirado em plataformas de streaming e upload como Spotify e SoundCloud, sem vínculo comercial ou institucional com essas empresas.

1.3 Convenções utilizadas

Nomes de rotas, tabelas, variáveis e arquivos são citados no formato de código (ex.: `server.js`). Requisitos funcionais são identificados pelo prefixo RF; requisitos não funcionais, pelo prefixo RNF. Trechos entre colchetes indicam informação que deve ser confirmada ou preenchida pela equipe antes da publicação (ex.: `[confirmar]`).

2. VISÃO GERAL DO PRODUTO

2.1 Descrição geral

O OpenSound resolve o problema de compartilhamento simples e rápido de faixas musicais: qualquer usuário autenticado pode publicar uma música (com título, artista, arquivo de áudio e capa opcional) sem passar por um processo de aprovação prévia, e o catálogo é atualizado em tempo real para todos os visitantes. A proposta central é reduzir o atrito para novos artistas e ouvintes, priorizando facilidade de uso sobre curadoria.

2.2 Público-alvo

Artistas independentes e criadores de conteúdo que desejam publicar faixas sem burocracia; disponibilizar a oportunidade de publicar faixas específicas que existem em apenas um streaming, como por exemplo versões de músicas ao vivo ou covers; ouvintes que buscam descobrir músicas recentes; e, no contexto atual do projeto, avaliadores acadêmicos e colaboradores da equipe de desenvolvimento.

2.3 Posicionamento em relação a referências de mercado

Diferente do Spotify (modelo majoritariamente fechado, com catálogo curado por selos e distribuidoras), o OpenSound segue o modelo aberto de upload direto do SoundCloud, no qual o próprio usuário é responsável pelo conteúdo publicado. Essa escolha de produto tem uma implicação jurídica direta, tratada em detalhe no Termo de Licenciamento e Uso: a plataforma não realiza checagem prévia de direitos autorais sobre as faixas enviadas.

2.4 Estado atual do projeto

O sistema encontra-se em estágio de MVP (Produto Mínimo Viável) funcional: cadastro com verificação em duas etapas, login, upload de músicas e listagem pública já estão implementados e operantes. Funcionalidades como histórico de escuta, monetização e perfis de artista estão planejadas para etapas futuras

3. ARQUITETURA DO SISTEMA

3.1 Visão macro

O OpenSound segue uma arquitetura cliente-servidor de três camadas:

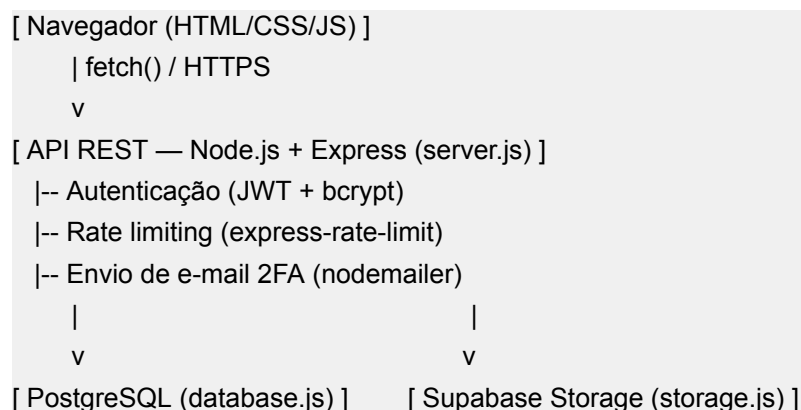
- Camada de apresentação (frontend): HTML, CSS e JavaScript vanilla (sem framework), executados no navegador do usuário.
- Camada de aplicação (backend): API REST em Node.js com Express, responsável pelas regras de negócio, autenticação e validação.
- Camada de dados: banco relacional PostgreSQL para dados estruturados (usuários, músicas, verificações) e Supabase Storage (armazenamento de objetos) para os arquivos binários de áudio e capa.

Fluxo simplificado: o navegador consome a API REST via `fetch()` (MDN WEB DOCS, 2026); o backend valida a requisição, aplica autenticação/autorização quando necessário, consulta ou grava no PostgreSQL e, quando há arquivo, envia o binário ao Supabase Storage, devolvendo ao frontend apenas metadados e URLs públicas.

3.2 Diagrama textual de arquitetura

A Figura 1 ilustra a arquitetura do sistema.

Figura 1 - Arquitetura do sistema em três camadas



usuarios / musicas /	bucket "musicas"
verificacoes_2fa	audio/ e capas/

Fonte: Elaborado pelos autores (2026).

3.3 Stack tecnológica

O Quadro 1 apresenta as tecnologias empregadas em cada camada do sistema e sua função.

Tabela 1 - Pilha tecnológica do OpenSound

Camada	Tecnologia	Função no projeto
Frontend	HTML5 / CSS3 / JavaScript (ES6+)	Interface, modais, player, perfil e chamadas à API
Backend	Node.js + Express 5	Servidor HTTP e rotas da API REST
Banco de dados	PostgreSQL (pacote pg)	Persistência de usuários, músicas, playlists, perfil e verificações
Armazenamento	Supabase Storage (@supabase/supabase-js)	Arquivos de áudio, capas, avatares, banners e fundos de perfil
Autenticação	JWT (jsonwebtoken) + bcrypt	Token de sessão e hash de senha/códigos
E-mail	Nodemailer (SMTP Gmail)	Códigos de verificação (cadastro e recuperação de senha)
Upload	Multer (memória)	Recebe multipart/form-data e repassa ao Storage
Tráfego	express-rate-limit, cors	Limitação de requisições por IP e controle de origem
Configuração	dotenv	Variáveis de ambiente (.env)

Hospedagem	Vercel (frontend) e Render (backend)	Publicação do site e da API
Observabilidade	Módulo ajudantes.js	Logs de desenvolvimento (DEBUG_LOGS), validação de arquivos e funções auxiliares

Fonte: Elaborado pelos autores (2026).

3.4 Organização de diretórios

```

open-sound/
├── backend/
│   ├── server.js          → rotas da API e regras de negócio
│   ├── database.js        → conexão e criação/migração das tabelas
│   ├── storage.js         → integração com o Supabase Storage
│   ├── ajudantes.js       → logs, validação de arquivos e utilitários
│   ├── aplicar-migracao.js → executa migração do esquema isoladamente
│   ├── verificar-migracao.js → pós-migração somente leitura
│   ├── diagnostico-nome-usuario.js → diagnóstico de duplicatas de @
│   ├── pre-visualizacao-handles.js → simula derivação de handles
│   ├── .env.example       → modelo das variáveis de ambiente
│   └── package.json        → dependências do backend
├── frontend/
│   ├── html/              → home, biblioteca, playlist, perfil,
│   │                      minhas-musicas, config
│   ├── css/               → root, theme, sessão, home, perfil,
│   │                      modal-playlist, minhas-musicas
│   └── js/                 → sessão, player, home, biblioteca, playlist,
│                           favoritos, profile, minhas-musicas, config
├── project docs/          → política de privacidade, termos de uso,
│                           MVP, stack e dependências
└── README.md

```

Fonte: Elaborado pelos autores (2026).

4. MODELAGEM DE DADOS

O banco relacional é criado automaticamente na inicialização do servidor (função `criarTabelas` em `database.js`), por meio de instruções `CREATE TABLE IF NOT EXISTS`. Como essa instrução não altera tabelas já existentes, colunas adicionadas depois são aplicadas por migrações leves (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS`), executadas a cada inicialização sem apagar dados. As colunas de data usam `TIMESTAMP TZ`, com conversão automática das antigas, e o servidor só aceita requisições depois que o esquema está pronto, repetindo a criação em caso de falha transitória do banco.

4.1 Tabela usuarios

Tabela 2 - Estrutura da tabela usuarios

Campo	Tipo	Regras
id	SERIAL	Chave primária
email	VARCHAR(255)	Único, obrigatório
senha	VARCHAR(255)	Hash bcrypt – nunca texto puro
nome_usuario	VARCHAR(255)	Handle (@); único (LOWER), 3-28 chars [A-Za-z0-9._-]; obrigatório
nome_exibicao	TEXT	Nome livre de exibição; inicializado com nome_usuario no cadastro
verificado	BOOLEAN	Padrão FALSE; TRUE após o 2FA
eh_artista	BOOLEAN	Padrão FALSE; TRUE após registrar nome de artista
nome_artista	VARCHAR(255)	Nome usado como autor das músicas
bio	TEXT	Biografia do perfil (máx. 300 caracteres)
url_avatar	TEXT	URL pública do avatar
url_banner	TEXT	URL pública do banner de perfil

url_fundo	TEXT	URL pública do fundo de perfil
personalizacao	JSONB	Configurações livres de personalização; padrão {}
status_emoji	TEXT	Emoji do status visível no perfil
status_texto	TEXT	Texto do status
mostrar_ouvindo	BOOLEAN	Controla a exibição do "ouvindo agora"; padrão FALSE
tema	VARCHAR(10)	Preferência de tema ("light" ou "dark"); padrão "dark"
senha_redefinida_em	TIMESTAMPTZ	Data da última redefinição de senha
criado_em	TIMESTAMPTZ	Data/hora de criação

Fonte: Elaborado pelos autores (2026).

4.2 Tabela musicas

Tabela 3 - Estrutura da tabela musicas

Campo	Tipo	Regras
id	SERIAL	Chave primária
titulo	VARCHAR(255)	Obrigatório
artista	VARCHAR(255)	Obrigatório; copiado de usuarios.nome_artista
url_audio	TEXT	URL pública do áudio; obrigatório
url_capa	TEXT	URL pública da capa; pode ser nula
usuario_id	INTEGER	FK -> usuarios(id)
reproducoes	INTEGER	Contador de reproduções; padrão 0
criado_em	TIMESTAMPTZ	Ordena o catálogo (mais recentes primeiro)

Fonte: Elaborado pelos autores (2026).

4.3 Tabelas de verificação temporária

Tabela 4 - Tabelas de verificação temporária

Tabela	Finalidade	Campos específicos
--------	------------	--------------------

verificacoes_2fa	Cadastro pendente (id UUID)	email, senha_hash, nome_usuario, nome_exibicao, codigo_hash
recuperacoes_senha	"Esqueci minha senha" (sem login)	email, codigo_hash, verificado (TRUE após validar o código)
redefinicoes_senha	Troca de senha por usuário logado	usuario_id (FK), nova_senha_hash, codigo_hash

Fonte: Elaborado pelos autores (2026).

4.4 Tabelas de histórico, playlists, perfil e player

Tabela 5 - Tabelas de histórico, playlists, perfil e player

Tabela	Finalidade	Campos principais
historico_redefinicoes_senha	Registra cada redefinição concluída; base do limite de 3 em 24 h	usuario_id (CASCADE), origem, criado_em
playlists	Playlists do usuário (máx. 20); "Favoritos" é única por usuário	nome, url_capa, usuario_id, eh_favoritos, publica
playlist_musicas	Junção playlist-música (sem duplicatas)	playlist_id, musica_id (CASCADE), UNIQUE
perfil_favorita	Música favorita do perfil (máx. uma por usuário)	usuario_id (PK), musica_id (CASCADE), atualizado_em
perfil_curtidas	Até 4 músicas em destaque no perfil	usuario_id, musica_id (CASCADE), posicao, UNIQUE(usuario_id, musica_id)
user_player_state	Estado do player e fila de reprodução por usuário	user_id (PK), current_track_id, progress_ms, is_playing, queue (JSONB), updated_at

Fonte: Elaborado pelos autores (2026).

4.5 Relacionamentos e integridade

musicas.usuario_id referencia usuarios.id, associando cada faixa ao seu autor. As tabelas de verificação são independentes: o registro definitivo em usuarios só é criado após a confirmação do código, evitando cadastros incompletos. Nesta revisão foi criado o índice único uq_usuarios_nome_usuario sobre LOWER(nome_usuario), resolvendo a pendência apontada no checklist da versão anterior (seção 13.2, item i). Tabelas dependentes usam ON DELETE CASCADE, e a exclusão de conta é feita em transação.

5. SEGURANÇA E AUTENTICAÇÃO

5.1 Cadastro com verificação em duas etapas (2FA)

O cadastro não cria o usuário imediatamente. O fluxo é:

- a) o usuário envia e-mail, @ (nome_usuario) e nome de exibição (nome_exibicao) e senha para POST /api/registro;
- b) o backend valida o @ (3-28 chars, apenas [A-Za-z0-9._-]) e a unicidade insensível à caixa, além de formato de e-mail e força da senha, antes de qualquer gravação; gera um código numérico de 6 dígitos e grava o hash na tabela verificacoes_2fa com expiração de 10 minutos;
- c) o código é enviado por e-mail via Nodemailer;
- d) o usuário confirma o código em POST /api/validar-2fa; ao atingir 5 tentativas incorretas, a verificação é descartada e um novo cadastro deve ser iniciado;
- e) somente após a confirmação o registro definitivo é criado em usuarios, com nome_usuario e nome_exibicao separados. Se outro cadastro reservou o mesmo @ no intervalo, o processo é recusado com código NOME_USUARIO_JA_EM_USO.

A senha deve ter 8 ou mais caracteres, com maiúscula, minúscula, número e ao menos um caractere especial do conjunto @ () ! % * ? & #; outros caracteres são recusados pela expressão regular atual.

5.2 Login e emissão de token (JWT)

O login (POST /api/login) confere e-mail e senha (comparação de hash com bcrypt) e, em caso de sucesso, emite um JSON Web Token (JSON WEB TOKEN, 2026) assinado com JWT_SECRET_SESSAO, carregando apenas dados de identificação

do usuário, nunca o hash da senha. A validade é de 7 dias por padrão ou 30 dias com a opção lembrarDeMim. Rotas protegidas exigem o cabeçalho Authorization: Bearer <token>, validado pelo middleware verificarAutenticacao.

5.3 Hash de senhas

Senhas nunca são armazenadas em texto puro: o backend usa bcrypt com fator de custo 10 tanto para a senha de conta quanto para o código de verificação 2FA, o que impede a recuperação do valor original mesmo em caso de vazamento do banco de dados.

5.4 Recuperação e redefinição de senha

Há dois fluxos de troca de senha:

- a) esqueci minha senha (sem login): esqueci-senha gera o código, validar-esqueci marca a recuperação como verificada e confirmar-esqueci grava a nova senha. Limite de 3 redefinições concluídas por conta em 24 horas, recheado na confirmação para impedir burla por várias sessões;
- b) redefinição logada (página de configurações): solicitar e confirmar exigem token e código enviado por e-mail.

5.5 Limitação de requisições (rate limiting)

Tabela 6- Limites de requisições por rota

Rota	Janela	Limite por IP
POST /api/registro	15 minutos	5
POST /api/validar-2fa	15 minutos	10
POST /api/reenviar-2fa	1 hora (+ cooldown 60 s)	5
POST /api/login	15 minutos	10
POST /api/auth/esqueci-senha	15 minutos	5

POST/api/auth/validar-esqueci e confirmar-esqueci	15 minutos	15
POST/api/auth/reenviar-esqueci	1 hora	5
POST/api/usuarios/redefinir-senha/*	15 minutos	10
POST /api/musicas (upload)	1 hora	20
POST /api/perfil/avatar	1 hora	30
POST /api/perfil/banner	1 hora	limitarBannerIP
GET/api/perfil/publico/:identificador	Configurável	limitarPerfilPublicoIP
GET /api/perfil/buscar	Configurável	limitarBuscaPerfilIP

Fonte: Elaborado pelos autores (2026).

5.6 Armazenamento de arquivos

Arquivos de áudio e imagem são recebidos em memória pelo Multer (nunca gravados em disco local, o que mantém o backend compatível com ambientes de hospedagem efêmeros) e repassados ao bucket "musicas" do Supabase Storage, dentro das pastas audio/ e capas/. Cada arquivo recebe um prefixo único (timestamp + número aleatório) para evitar colisão de nomes. Limite de 25 MB por arquivo; tipos aceitos: áudio MP3/WAV/OGG e imagem JPEG/PNG/WEBP. O tipo de imagem e de áudio é conferido pelo conteúdo do arquivo (assinatura binária), e não apenas pelo tipo informado pelo cliente.

5.7 Exclusão de conta

DELETE /api/usuarios/eu remove os arquivos do usuário no Storage (incluindo avatar, banner e fundo) e, em transação, apaga músicas, playlists, redefinições pendentes e a conta, com ROLLBACK em caso de erro. Atende ao direito de eliminação previsto na LGPD (BRASIL, 2018).

6. REQUISITOS FUNCIONAIS (MVP)

Tabela 7 - Requisitos funcionais

Código	Requisito	Status
RF01	Cadastro de usuário com e-mail, @ único (nome_usuario) e nome de exibição (nome_exibicao)	Implementado
RF02	Verificação em duas etapas (2FA) por e-mail no cadastro	Implementado
RF03	Reenvio do código 2FA, respeitando cooldown	Implementado
RF04	Login com e-mail e senha, com opção "lembrar de mim"	Implementado
RF05	Logout, encerrando a sessão local	Implementado
RF06	Upload de música (título, áudio, capa opcional) apenas para usuários autenticados que sejam artistas	Implementado
RF07	Listagem pública do catálogo, mais recentes primeiro	Implementado
RF08	Exigir login para reproduzir (navegar é livre; tocar não)	Implementado
RF09	Alternância de tema claro/escuro, com a preferência salva na conta	Implementado
RF10	Busca por música e artista	Implementado
RF11	Recomendações: músicas e artistas mais ouvidos	Implementado
RF12	Criação e organização de playlists (criar, listar, excluir, adicionar/remover músicas, capa)	Implementado (sem hierarquia)
RF13	Perfil de usuário com foto, banner, fundo, bio, status, nome de exibição, música favorita, músicas em destaque e playlists públicas	Implementado
RF14	Recuperação de senha ("esqueci minha senha") por código enviado ao e-mail	Implementado
RF15	Redefinição de senha por usuário logado, nas configurações	Implementado
RF16	Cadastro de nome de artista para habilitar uploads	Implementado
RF17	Contagem de reproduções por música	Implementado
RF18	Favoritar músicas (playlist automática "Favoritos")	Implementado
RF19	Listar e excluir as próprias músicas ("Minhas Músicas")	Implementado

RF20	Exclusão da conta e de todos os dados associados	Implementado
RF21	Filtro por gênero	Backlog
RF22	Histórico de escuta	Backlog
RF23	Player global persistente, com fila de reprodução que sobrevive à navegação e ao recarregamento	Implementado
RF24	Página "Minha Biblioteca" com playlists e favoritos	Implementado
RF25	Perfil público acessível por @ ou id numérico, sem autenticação obrigatória	Implementado
RF26	Busca pública de perfis por @, nome de exibição ou nome de artista	Implementado
RF27	Status de usuário (emoji + texto) e opção "ouvindo agora"	Implementado

Fonte: Elaborado pelos autores (2026).

7. API REST - REFERÊNCIA DE ENDPOINTS

7.1 Autenticação e cadastro

Tabela 8 - Endpoints de autenticação e cadastro

Método e rota	Auth	Descrição
POST /api/registro	Não	Corpo: email, password, nomeUsuario, nomeExibicao. Retorna idVerificacao
POST /api/validar-2fa	Não	Corpo: codigo, idVerificacao. Cria e ativa a conta
POST /api/reenviar-2fa	Não	Corpo: idVerificacao. Novo código (cooldown 60 s)
POST /api/login	Não	Corpo: email, password, lembrarDeMim. Retorna tokenSessao
POST /api/auth/esqueci-senha	Não	Inicia a recuperação e envia o código
POST /api/auth/reenviar-esqueci	Não	Reenvia o código de recuperação
POST /api/auth/validar-esqueci	Não	Valida o código de recuperação
POST /api/auth/confirmar-esqueci	Não	Grava a nova senha (limite de 3 em 24 h)

Fonte: Elaborado pelos autores (2026).

7.2 Usuário e perfil

Tabela 9 - Endpoints de usuário e perfil

Método e rota	Auth	Descrição
GET /api/usuarios/eu	Sim	Dados da conta logada
POST /api/usuarios/artista	Sim	Corpo: nomeArtista. Habilita upload
GET /api/perfil	Sim	Perfil completo do usuário logado

GET /api/perfil/publico/:identificador	Opcional	Perfil público por @ ou id numérico
GET /api/perfil/buscar?q=	Não	Busca pública por @, nome de exibição ou nome de artista
PUT /api/perfil/exibicao	Sim	Atualiza o nome de exibição (nome_exibicao)
PUT /api/perfil/bio	Sim	Biografia (até 300 caracteres)
POST /api/perfil/avatar	Sim	multipart, campo avatar (até 2 MB)
DELETE /api/perfil/avatar	Sim	Remove o avatar
POST /api/perfil/banner	Sim	multipart, campo banner (JPEG/PNG/WEBP/GIF)
DELETE /api/perfil/banner	Sim	Remove o banner
PUT /api/perfil/personalizacao	Sim	Salva configurações livres de personalização (JSONB)
PUT /api/perfil/status	Sim	Atualiza emoji, texto do status e/ou mostrarOuvindo
PUT /api/perfil/tema	Sim	Salva o tema ("light" ou "dark")
PUT e DELETE /api/perfil/favorita	Sim	Define ou remove a música favorita
PUT /api/perfil/curtidas	Sim	Define até 4 músicas em destaque no perfil
POST /api/perfil/playlists	Sim	Cria playlist pelo perfil
DELETE /api/perfil/playlists/:id	Sim	Exclui playlist pelo perfil
POST /api/usuarios/redefinir-senha/solicitar	Sim	Envia código para trocar a senha
POST /api/usuarios/redefinir-senha/confirmar	Sim	Confirma o código e grava a nova senha
DELETE /api/usuarios/eu	Sim	Exclui a conta e seus dados

Fonte: Elaborado pelos autores (2026).

7.3 Músicas

Tabela - 10 Endpoints de músicas

Método e rota	Auth	Descrição
POST /api/musicas	Sim (artista)	multipart: título, audio, capa (opcional)
GET /api/musicas	Não	50 músicas mais recentes
GET /api/musicas/mais-tocadas	Não	10 músicas com mais reproduções
GET /api/musicas/buscar?q=	Não	Busca por título/artista (até 50)
GET /api/artistas/mais-ouvidos	Não	10 artistas por reproduções
GET /api/musicas/minhas	Sim	Músicas do usuário logado
POST /api/musicas/:id/reproduzir	Sim	Incrementa o contador de reproduções
DELETE /api/musicas/:id	Sim	Exclui música própria

Fonte: Elaborado pelos autores (2026).

7.4 Playlists e favoritos

Tabela 11- Endpoints de playlists e favoritos

Método e rota	Auth	Descrição
GET /api/playlists	Sim	Lista playlists com total de músicas

POST /api/playlists	Sim	Cria playlist (nome; capa opcional até 5 MB)
GET /api/playlists/:id	Sim	Detalhe e músicas da playlist
DELETE /api/playlists/:id	Sim	Exclui playlist
POST /api/playlists/:id/musicas	Sim	Corpo: musicalId
DELETE /api/playlists/:id/musicas/:musicalId	Sim	Remove música da playlist
POST e DELETE /api/musicas/:id/favoritar	Sim	Adiciona/remove de Favoritos
GET /api/musicas/favoritos/ids	Sim	IDs das músicas favoritadas

Fonte: Elaborado pelos autores (2026).

7.5 Player e fila de reprodução

Tabela 12 - Endpoints do player e da fila

Método e rota	Auth	Descrição
GET /api/player/state	Sim	Estado salvo: faixa atual, posição, reprodução e fila
PUT /api/player/state	Sim	Salva o estado do player
POST /api/player/queue	Sim	Adiciona música à fila
PATCH /api/player/queue/reorder	Sim	Reordena a fila
DELETE /api/player/queue/:trackId	Sim	Remove uma música da fila

Fonte: Elaborado pelos autores (2026).

8. FRONTEND - ESTRUTURA, TAGS E COMPONENTES

O frontend não usa framework: é HTML, CSS e JavaScript organizados por responsabilidade. Esta seção documenta as principais tags e elementos de `home.html`, explicando o papel de cada bloco, útil tanto para manutenção quanto para quem for integrar novas telas seguindo o mesmo padrão.

8.1 Barra de navegação (<nav class="fixed-bar">)

- `nav-left`: botões de navegação principal — "Minha Biblioteca", "Início" e "Upload".
- `nav-center`: formulário de busca (<form action="/buscar">) com campo <input type="search">.
- `nav-right`: botões de conta - "registre-se", "Entrar", "Sair" (oculto via classe `.hidden` quando deslogado) e alternância de tema.

8.2 Conteúdo principal (<div class="main-ctt">)

- `#lista-musicas`: contêiner universal preenchido dinamicamente pelo JavaScript a partir de GET `/api/musicas` - cada música vira um "card" (capa, título, artista, botão de tocar).
- `.double-div`: agrupa duas áreas ainda estáticas/planejadas - `.artists` (artistas mais ouvidos) e `.genres` (filtro por gênero).

8.3 Modais (<div class="modal">)

- `#modal-registro`: modal de duas etapas - `#etapa-registro` (formulário de e-mail/senha, com lista de regras de senha em tempo real via <ul id="regras-senha">) e `#etapa-2fa` (grade de 6 <input class="input-otp"> com foco automático e suporte a colar o código completo).
- `#modal-login`: formulário simples de e-mail, senha e checkbox "lembrar de mim por 30 dias".
- `#modal-upload`: formulário de envio de música com pré-visualização ao vivo (`preview-upload-titulo`, `preview-upload-artista`, `preview-upload-capa`) que é atualizada a cada tecla digitada ou arquivo selecionado, antes mesmo do envio ao servidor.

8.4 Player fixo (<div id="player-barra">)

Barra fixa na base da tela, oculta até a primeira reprodução. Contém capa, título, artista, botão de play/pause, tempo decorrido, uma barra de progresso (<input type="range" id="player-seek">) e o tempo total da faixa - controlada inteiramente pelo home.js a partir de um elemento <audio> HTML5 gerenciado em memória.

8.5 Nomenclatura

IDs e classes seguem nomenclatura em português, alinhada ao restante do código (ex.: btn-entrar, campo-senha, mensagem-lista). Manter esse padrão em novas telas evita inconsistência entre arquivos.

9. FLUXOS DE USO (CASOS DE USO)

9.1 Cadastro de novo usuário

- a) usuário clica em "registre-se" e preenche e-mail, @ (nome_usuario, 3-28 chars alfanumérico), nome de exibição (nome_exibicao, livre) e senha (regras validadas em tempo real);
- b) o sistema valida unicidade do @ antes de enviar o código de 6 dígitos por e-mail;
- c) usuário digita ou cola o código; o sistema confirma, verifica novamente a unicidade do @ e ativa a conta;
- d) alternativa: se o e-mail já existir, o cadastro é interrompido e abre-se o modal de login pré-preenchido.

9.2 Login e recuperação de senha

- a) usuário informa e-mail e senha, opcionalmente marca "lembrar de mim"; o token de sessão é guardado no navegador;
- b) se esqueceu a senha, abre o modal de recuperação, valida o código enviado por e-mail e define nova senha.

9.3 Tornar-se artista e enviar música

- a) usuário autenticado clica em "Upload"; se ainda não for artista, informa o nome de artista;
- b) preenche título, seleciona áudio e capa, acompanhando a pré-visualização;
- c) o backend valida, grava os arquivos no Storage e cria o registro; o catálogo é recarregado.

9.4 Reprodução, favoritos e playlists

- a) qualquer visitante navega pelo catálogo; ao clicar em "tocar", sem sessão ativa, o login é solicitado;
- b) ao tocar, a reprodução é contabilizada em POST /api/musicas/:id/reproduzir;
- c) no player, o usuário pode favoritar a faixa ou adicioná-la a uma playlist.

9.5 Gestão de perfil e conta

- a) em "perfil", o usuário edita avatar, banner, bio, status (emoji + texto + ouvindo agora), nome de exibição, música favorita e músicas em destaque;
- b) o perfil pode ser acessado publicamente via URL /perfil/@handle ou /perfil/id;
- c) em "Minhas Músicas", exclui faixas próprias; em "Configurações", redefine a senha ou exclui a conta.

10. REQUISITOS NÃO FUNCIONAIS

Tabela 13 - Requisitos não funcionais

Código	Categoria	Descrição
RNF01	Segurança	Senhas e códigos nunca armazenados em texto puro (hash bcrypt)
RNF02	Segurança	Toda rota sensível exige token JWT válido, validado no backend
RNF03	Disponibilidade	Limite de requisições por IP para reduzir risco de abuso
RNF04	Usabilidade	Erros em linguagem clara, em português, em todas as chamadas da API
RNF05	Portabilidade	Backend sem dependência de armazenamento em disco local
RNF06	Manutenibilidade	Separação entre rotas (server.js), dados (database.js) e arquivos (storage.js)
RNF07	Conformidade legal	Tratamento de dados pessoais alinhado à LGPD (Lei nº 13.709/2018)
RNF08	Integridade	Migrações de esquema idempotentes, datas com fuso (TIMESTAMPZ) e exclusão de conta em transação
RNF09	Observabilidade	Logs de desenvolvimento ativados por DEBUG_LOGS; erros e avisos sempre registrados

Fonte: Elaborado pelos autores (2026).

11. INSTALAÇÃO E EXECUÇÃO (AMBIENTE DE DESENVOLVIMENTO)

11.1 Pré-requisitos

São necessários:

- a) Node.js instalado;
- b) instância de PostgreSQL acessível;
- c) projeto no Supabase, com Storage habilitado;
- d) conta Gmail para envio dos códigos via Nodemailer.

11.2 Passo a passo

- a) copiar .env.example para .env e preencher as credenciais;
- b) abrir o terminal na pasta backend;
- c) executar npm install;
- d) executar npm start (ou node server.js);
- e) abrir frontend/html/home.html no navegador (ou servir a pasta por servidor estático);
- f) encerrar o servidor com Ctrl + C.

Para verificar ou aplicar migrações do banco de dados separadamente, estão disponíveis os scripts auxiliares: node aplicar-migracao.js (executa a migração isoladamente) e node verificar-migracao.js (somente leitura, confirma que a migração foi aplicada).

11.3 Variáveis de ambiente necessárias

Tabela 14 - Variáveis de ambiente

Variável	Finalidade
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	Conexão com o PostgreSQL
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY	Acesso administrativo ao Storage (uso exclusivo do backend)
SUPABASE_BUCKET_MUSICAS	Nome do bucket (padrão: "musicas")
GMAIL_USER, GMAIL_PASS	Conta usada pelo Nodemailer
JWT_SECRET_SESSAO	Segredo dos tokens – obrigatório, sem valor padrão
PORT	Porta HTTP (padrão: 3000)
DEBUG_LOGS	Se "true", ativa os logs detalhados de desenvolvimento
SMTP_INSECURE_TLS	Se "true", desliga a verificação do certificado TLS do e-mail (somente desenvolvimento)

Fonte: Elaborado pelos autores (2026).

12. PERSPECTIVAS FUTURAS

Com base no planejamento de produto da equipe, os itens a seguir estão fora do escopo do MVP atual, mas mapeados para próximas etapas:

- Perfil de usuário completo: foto, biografia, banner e tema padrão.
- Playlists organizáveis, com hierarquia e fixação de itens.
- Histórico de escuta e filtros inteligentes de recomendação.
- Comentários em músicas e status de usuário (rede social leve).
- Selo de "artista verificado".
- Modelo de monetização: assinatura premium sem anúncios e repasse percentual de receita aos artistas mais ouvidos.
- Relatório periódico de estatísticas de escuta para o usuário (retrospectiva anual, minutos ouvidos, artistas mais tocados).

13. REFERÊNCIAS

ANTHROPIC. **Claude**. Versão 5.5 Sonnet. San Francisco: Anthropic, 2026. Disponível em: <https://claude.ai>. Acesso em: 30 set. 2026.

BCRYPT. **bcrypt**. [S. l.]: npm, 2026. Disponível em: <https://www.npmjs.com/package/bcrypt>. Acesso em: 30 set. 2026.

BRASIL. **Lei nº 9.610, de 19 de fevereiro de 1998**. Altera, atualiza e consolida a legislação sobre direitos autorais e dá outras providências. Brasília, DF: Presidência da República, 1998. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l9610.htm. Acesso em: 30 de set. 2026.

BRASIL. **Lei nº 12.965, de 23 de abril de 2014**. Marco Civil da Internet. Brasília, DF: Presidência da República, 2014. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2011-2014/2014/lei/l12965.htm. Acesso em: 30 set. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 30 set. 2026.

EXPRESS.JS. **Express**: Node.js web application framework. [S. l.], 2026. Disponível em: <https://expressjs.com>. Acesso em: 30 set. 2026.

JSON WEB TOKEN. **Introduction to JSON Web Tokens**. [S. l.], 2026. Disponível em: <https://jwt.io/introduction>. Acesso em: 29 set 2026.

NODEMAILER. **Nodemailer**: send emails from Node.js. [S. l.], 2026. Disponível em: <https://nodemailer.com>. Acesso em: 29 set. 2026.

OPENAI. **ChatGPT**. Versão GPT-4o. San Francisco: OpenAI, 2026. Disponível em: <https://chatgpt.com>. Acesso em: 29 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation**. [S. l.], 2026. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 30 set. 2026.

RENDER. **Render documentation**. [S. l.], 2026. Disponível em: <https://render.com/docs>. Acesso em: 30 set. 2026.

SUPABASE. **Storage**. [S. l.], 2026. Disponível em: <https://supabase.com/docs/guides/storage>. Acesso em: 30 set. 2026.

VERCEL. **Vercel documentation**. [S. l.], 2026. Disponível em: <https://vercel.com/docs>. Acesso em: 30 set. 2026.